

LAPORAN PROJECT EAS
PEMROGRAMAN BERORIENTASI OBJEK_14823274



Nama Anggota:

- 1. Muhammad Fahmi Ardiansyah (1482500038)**
- 2. Aditya Edgar Saputra (1482500039)**

KELAS: B

Dosen Pengampu

Bagus Hardiansyah, S.kom., M.si.

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI

JUNI 2026

Daftar Isi

Daftar Isi	2
KATA PENGANTAR	4
BAB I: PENDAHULUAN.....	5
1.1 Latar Belakang	5
1.2 Rumusan Masalah.....	5
1.3 Batasan Masalah	6
1.4 Tujuan Project	6
BAB II LANDASAN TEORI.....	7
2.1 Pemrograman Berorientasi Objek (OOP)	7
2.1.1 Class dan Objek:	7
2.1.2 Enkapsulasi (Encapsulation):	7
2.1.3 Modularitas (Modularity):.....	7
2.2 Class Method pada Python	7
2.3 Manajemen Memori In-Memory Database	7
BAB III METODOLOGI PENELITIAN	8
3.1 Tahapan Pelaksanaan Proyek	8
3.2 Analisis Kebutuhan Sistem (Requirement Analysis).....	8
3.2.1 Aktor Mahasiswa:.....	8
3.2.2 Aktor Administrator:	8
3.3 Desain Aturan Kodifikasi dan Arsitektur PBO.....	8
3.4 Metode Pengujian Sistem	9
BAB IV ANALISIS DAN PERANCANGAN SISTEM	10
4.1 Analisis Alur Sistem	10
4.1.1 Fungsi Registrasi Mandiri:	10
4.1.2 Fungsi Otentikasi Terintegrasi:	10
4.1.3 Fungsi Agregasi Administrasi:.....	10
4.2 Desain Kode Struktur Identitas (NIM)	10
4.3 Struktur Modular File	11
4.3.1 pendaftaran.py:.....	11
4.3.2 penjadwalan.py:.....	11
4.3.3 main.py:	11
BAB V IMPLEMENTASI DAN PEMBAHASAN KODE.....	12

5.1 Modul Entitas Mahasiswa (pendaftaran.py)	12
5.2 Modul Manajer Kurikulum Akademik (penjadwalan.py)	13
5.3 Arsitektur Modul Utama (main.py)	14
BAB VI EVALUASI DAN PENGUJIAN SISTEM	15
6.1 Matriks Skenario Uji (Black Box Testing)	15
6.2 Dokumentasi Hasil Eksekusi Terminal Visual Studio Code	16
6.2.1 Log Tampilan Pendaftaran (Pilihan Menu 1)	16
6.2.2 Log Tampilan Halaman KRS Mahasiswa (Pilihan Menu 2)	17
6.2.3 Log Tampilan Menu Admin (Pilihan Menu 3)	17
BAB VII PENUTUP	18
7.1 Kesimpulan	18
7.2 Saran	18
DAFTAR PUSTAKA	19

KATA PENGANTAR

Puji syukur penulis panjatkan kehadirat Tuhan Yang Maha Esa atas segala rahmat, hidayah, dan karunia-Nya, sehingga penulis dapat menyelesaikan proyek berkelompok beserta laporan praktikum ini yang berjudul "**Sistem Pendaftaran Mahasiswa & Penjadwalan Mata Kuliah**" dengan tepat waktu dan tanpa kendala struktural yang berarti. Laporan ini disusun sebagai salah satu instrumen penilaian akademis yang komprehensif pada mata kuliah Pemrograman Berorientasi Objek. Fokus utama dari pengerjaan proyek ini adalah menyimulasikan proses bisnis berskala mikro pada tata kelola administrasi perguruan tinggi, yang meliputi manajemen registrasi, penentuan Nomor Induk Mahasiswa (NIM) terenkapsulasi, otentikasi dinamis, hingga pemetaan kurikulum paket semester awal secara modular.

Penulis menyampaikan rasa terima kasih yang sebesar-besarnya kepada Dosen Pengampu mata kuliah Pemrograman Berorientasi Objek atas bimbingan teoritis dan praktis yang telah diberikan sepanjang perkuliahan. Penulis juga mengapresiasi dukungan moral serta diskusi konstruktif bersama rekan-rekan mahasiswa program studi Sistem dan Teknologi Informasi Universitas 17 Agustus 1945 Surabaya selama proses perancangan dan *debugging* arsitektur program ini. Penulis menyadari bahwa sistem ini masih memerlukan eskalasi fungsi lebih lanjut, sehingga segala bentuk kritik dan masukan akademis akan diterima demi penyempurnaan kompetensi di masa mendatang.

Surabaya, Juni 2026

BAB I: PENDAHULUAN

1.1 Latar Belakang

Efisiensi tata kelola data akademik merupakan salah satu parameter utama keberhasilan operasional lembaga pendidikan tinggi. Proses penerimaan mahasiswa baru, pembentukan identitas unik berupa Nomor Induk Mahasiswa (NIM), dan distribusi Kartu Rencana Studi (KRS) pada semester awal sering kali menghadapi kendala administratif jika sistem yang digunakan masih bersifat parsial atau tidak terintegrasi secara modular. Penanganan data berskala besar menuntut integritas data yang tinggi guna meminimalkan risiko terjadinya duplikasi informasi dan inkonsistensi *state* data pada memori aplikasi.

Pendekatan Pemrograman Berorientasi Objek (PBO) memberikan solusi arsitektural yang andal untuk menangani masalah tersebut. Melalui teknik pembagian tanggung jawab (*separation of concerns*) ke dalam modul-modul independen, data mahasiswa dapat diabstraksikan menjadi objek-objek diskret yang memiliki karakteristik (*attributes*) dan perilaku (*methods*) sendiri.

Proyek akhir ini berfokus pada pengembangan purwarupa sistem informasi akademik interaktif berbasis *Command Line Interface* (CLI) dengan bahasa Python di lingkungan Visual Studio Code. Aplikasi ini dirancang menggunakan arsitektur modular terpisah yang membagi sistem menjadi komponen pengontrol menu, komponen entitas mahasiswa yang terenkapsulasi ketat, serta komponen utilitas kurikulum akademik. Pemanfaatan struktur data *in-memory* berbasis *hash map* digunakan untuk memastikan performa pencarian data tetap optimal pada saat proses otentikasi dan agregasi statistik berlangsung.

1.2 Rumusan Masalah

1. Bagaimana mengimplementasikan prinsip modularitas untuk memisahkan fungsi kontrol program, entitas data mahasiswa, dan sistem penjadwalan kurikulum ke dalam berkas kode yang terpisah?
2. Bagaimana merancang mekanisme pembentukan NIM 10 digit secara otomatis yang memiliki sifat *information hiding* agar tidak dapat dimanipulasi secara eksternal?
3. Bagaimana mengotomatisasikan penghitungan akumulasi SKS secara dinamis serta mendistribusikan paket matkul secara instan berdasarkan keterkaitan program studi mahasiswa setelah proses otentikasi dinyatakan valid?

1.3 Batasan Masalah

1. Pengembangan aplikasi menggunakan bahasa pemrograman Python 3.x dan dieksekusi melalui terminal bawaan Visual Studio Code.
2. Penyimpanan data bersifat temporer di dalam memori runtime program (*in-memory database*) menggunakan struktur data *dictionary*.
3. Fakultas yang disediakan dibatasi pada FTEIC, TEKNIK, dan FEB, dengan program studi yang telah ditentukan pada masing-masing fakultas induk.

1.4 Tujuan Project

1. Merancang dan mengimplementasikan arsitektur modular perangkat lunak yang terbagi atas modul kontrol (*main.py*), modul blueprint objek mahasiswa (*pendaftaran.py*), dan modul manajer kurikulum (*penjadwalan.py*).
1. Menerapkan konsep *private method* dengan *name mangling* untuk mengamankan fungsi pembangkit NIM otomatis dari intervensi luar.
2. Mengaplikasikan *class method* (*@classmethod*) pada pengelolaan paket mata kuliah guna meminimalkan alokasi memori berulang untuk data yang bersifat statis.

BAB II

LANDASAN TEORI

2.1 Pemrograman Berorientasi Objek (OOP)

Pemrograman Berorientasi Objek (OOP) adalah paradigma yang menggunakan "objek" sebagai entitas dasar pembangun sistem dengan mengelompokkan data dan fungsi terkait dalam satu unit. Tiga prinsip dasar OOP yang diterapkan dalam proyek ini meliputi:

2.1.1 Class dan Objek: Class merupakan cetak biru (blueprint) yang mendefinisikan struktur data, sedangkan objek (instance) adalah wujud nyatanya saat dialokasikan di dalam RAM.

2.1.2 Enkapsulasi (Encapsulation): Prinsip menyembunyikan detail internal kelas dan hanya menyediakan antarmuka publik yang aman. Pada Python, hal ini dicapai lewat awalan double underscore (`__`) untuk memicu mekanisme *name mangling*.

2.1.3 Modularitas (Modularity): Pemisahan kode menjadi modul independen guna meminimalkan keterikatan antar-komponen (*low coupling*) dan meningkatkan keterpaduan fungsi di dalamnya (*high cohesion*).

2.2 Class Method pada Python

Class method ditandai dengan dekorator `@classmethod`. Berbeda dari instance method yang menerima referensi objek (`self`), class method menerima referensi dari kelas itu sendiri (`cls`) sebagai argumen pertama. Metode ini ideal untuk mengelola data global pada level kelas (*class-level state*) tanpa perlu instansiasi objek baru, sehingga menghemat alokasi memori runtime.

2.3 Manajemen Memori In-Memory Database

In-memory database memanfaatkan RAM sebagai media penyimpanan utama selama aplikasi berjalan. Pada Python, implementasi ini menggunakan objek dictionary (`{}`) berbasis pasangan Key-Value. Memanfaatkan mekanisme tabel pencarian *hash table*, operasi penambahan (*insertion*), penghapusan (*deletion*), dan pencarian data memiliki kompleksitas waktu konstan $O(1)$, tidak terpengaruh oleh jumlah data mahasiswa yang tersimpan di dalam sistem.

BAB III METODOLOGI PENELITIAN

3.1 Tahapan Pelaksanaan Proyek

Metodologi pelaksanaan dan perancangan proyek perangkat lunak ini didasarkan pada siklus pengembangan yang sistematis. Alur pengerjaan dimodelkan secara linear-interaktif untuk memastikan bahwa setiap fungsi yang dibangun memenuhi spesifikasi akademik:

[Analisis Kebutuhan] —> [Aturan Kodifikasi] —> [Pemodelan Modular PBO] —> [Pengujian Black Box]

3.2 Analisis Kebutuhan Sistem (Requirement Analysis)

Tahap awal dilakukan dengan memetakan kebutuhan fungsional dari aplikasi administrasi mikro ini. Skenario penggunaan dibagi menjadi dua aktor utama:

3.2.1 Aktor Mahasiswa: Memerlukan fungsi registrasi mandiri yang adaptif (pilihan program studi menyusut secara otomatis mengikuti fakultas induknya), serta hak akses masuk portal menggunakan padanan data NIM dan tanggal lahir untuk mencetak kurikulum paket secara instan.

3.2.2 Aktor Administrator: Memerlukan gerbang masuk terproteksi (*restricted area*) untuk memantau ringkasan total mahasiswa dan penyebaran datanya di setiap program studi tanpa mengganggu integritas objek mahasiswa itu sendiri.

3.3 Desain Aturan Kodifikasi dan Arsitektur PBO

Pada tahap ini, aturan pembentukan identitas unik dipatok menggunakan formula matematis string terikat dengan validasi keanggotaan *state map*. Formula kodifikasi NIM 10 digit dirancang sebagai berikut:

NIM = "26" (Tahun Masuk) + Kode Fakultas (1 Digit) + Kode Prodi (1 Digit)
+ Angka Acak (6 Digit)

Arsitektur sistem kemudian didekonstruksi menjadi tiga komponen modular berbasis objek: objek entitas (pendaftaran.py), objek pembawa kurikulum (penjadwalan.py), dan objek pengatur alur eksekusi menu (main.py).

3.4 Metode Pengujian Sistem

Untuk memvalidasi fungsionalitas sistem yang telah dibangun, proyek ini menerapkan metode Black Box Testing. Pengujian difokuskan sepenuhnya pada fungsionalitas masukan (*input*) dan keluaran (*output*) terminal aplikasi guna memastikan tidak ada logika percabangan yang mengalami kegagalan, kebocoran data antar-objek, ataupun penanganan string yang tidak konsisten saat program dijalankan di terminal Visual Studio Code.

BAB IV

ANALISIS DAN PERANCANGAN SISTEM

4.1 Analisis Alur Sistem

Aplikasi ini dirancang untuk memproses tiga fungsi operasional utama yang saling terikat:

4.1.1 Fungsi Registrasi Mandiri: Calon mahasiswa memasukkan data diri lengkap secara interaktif melalui terminal. Sistem menyaring pilihan fakultas dan secara adaptif memunculkan program studi yang relevan di bawah naungan fakultas tersebut.

4.1.2 Fungsi Otentikasi Terintegrasi: Akun mahasiswa langsung aktif setelah pendaftaran selesai. NIM yang terbentuk bertindak sebagai *username*, dan string tanggal lahir dengan format DDMMYYYY bertindak sebagai *password* rahasia.

4.1.3 Fungsi Agregasi Administrasi: Administrator internal kampus dibekali hak akses khusus untuk memantau ringkasan statistik total mahasiswa yang terdaftar serta sebarannya pada masing-masing program studi secara *real-time*.

4.2 Desain Kode Struktur Identitas (NIM)

Nomor Induk Mahasiswa disusun dari kombinasi string sepanjang 10 digit dengan aturan kodifikasi akademik yang baku. Susunan digit tersebut dimodelkan sebagai berikut:

$$\begin{aligned} \text{NIM} = & \underbrace{"26"}_{\text{Tahun Masuk (2 Digit)}} + \underbrace{\text{Kode Fakultas (1 Digit)}}_{\text{FTEIC=1, TEKNIK=2, FEB=3}} + \underbrace{\text{Kode Prodi (1 Digit)}}_{\text{Informatika=1, STI=2}} \\ & + \underbrace{\text{Pseudo-Random Number (6 Digit)}}_{\text{Range: 100000 s.d 999999}} \end{aligned}$$

Sistem dilengkapi dengan fungsi validasi perulangan logis (*while True*) untuk memeriksa apakah nilai NIM kandidat yang baru saja dibangkitkan sudah eksis di dalam penyimpanan atau belum. Jika terdeteksi konflik keanggotaan data, sistem akan memicu pembuatan ulang angka acak sampai diperoleh string unik.

4.3 Struktur Modular File

Project akhir ini dipecah ke dalam tiga berkas skrip Python yang bekerja secara sinergis:

4.3.1 pendaftaran.py: Modul entitas yang memuat definisi class Mahasiswa, deklarasi variabel instansiasi objek, serta penanganan alur masukan formulir biodata pendaftar.

4.3.2 penjadwalan.py: Modul layanan akademik yang memuat class KRS. Modul ini mengelola kamus data paket mata kuliah beserta bobot sks masing-masing program studi.

4.3.3 main.py: Berkas eksekusi utama (*entry point*) aplikasi yang mengendalikan alur menu interaktif, validasi otentikasi login pengguna, dan manajemen variabel *state dictionary* global.

BAB V

IMPLEMENTASI DAN PEMBAHASAN KODE

5.1 Modul Entitas Mahasiswa (pendaftaran.py)

Modul ini mendefinisikan konstruktor kelas dan menyembunyikan logika pembentukan NIM melalui *private method* `__generate_nim`.

```
1 import random
2
3 class Mahasiswa:
4     # Constructor untuk membuat objek Mahasiswa
5     def __init__(self, nama, jk, tempat_lahir, tgl_lahir, no_telp, alamat, email, warga, fakultas, prodi, db_mahasiswa):
6         self.nama = nama
7         self.jk = jk
8         self.tempat_lahir = tempat_lahir
9         self.tgl_lahir = tgl_lahir
10        self.no_telp = no_telp
11        self.alamat = alamat
12        self.email = email
13        self.warga = warga
14        self.fakultas = fakultas
15        self.prodi = prodi
16        self.password = tgl_lahir # Password otomatis menggunakan tanggal lahir
17        self.nim = self.__generate_nim(db_mahasiswa) # Private Method Enkapsulasi
18
19    # Private Method untuk generate NIM
20    def __generate_nim(self, db_mahasiswa):
21        tahun = "26"
22        kode_fakultas = {"FTEIC": "1", "TEKNIK": "2", "FEB": "3"}
23        kode_prodi = {
24            "Teknik Informatika": "1", "Sistem & Teknologi Informasi": "2",
25            "Teknik Arsitektur": "1", "Teknik Industri": "2",
26            "Manajemen": "1", "Akuntansi": "2"
27        }
28
29        fak_code = kode_fakultas.get(self.fakultas, "0")
30        prod_code = kode_prodi.get(self.prodi, "0")
31
32        while True:
33            random_num = str(random.randint(100000, 999999))
34            candidate_nim = tahun + fak_code + prod_code + random_num
35            if candidate_nim not in db_mahasiswa:
36                return candidate_nim
```

Pembahasan Analitis: Penggunaan parameter `db_mahasiswa` di dalam metode `__generate_nim` mencerminkan integrasi objek yang baik. Dengan melewati referensi basis data ke dalam *private method*, objek mampu memvalidasi keunikan dirinya sendiri sebelum atribut `self.nim` diisi dan dikunci secara permanen saat proses instansiasi selesai.

5.2 Modul Manajer Kurikulum Akademik (penjadwalan.py)

Mengelola koleksi data paket mata kuliah universitas secara efisien lewat pemanfaatan properti statis kelas dan dekorator `@classmethod`.

```
1 class KRS:
2     JADWAL_PAKET = {
3         "Teknik Informatika": [
4             {"matkul": "Dasar Pemrograman", "sks": 3},
5             {"matkul": "Kalkulus I", "sks": 3},
6             {"matkul": "Pengantar Teknologi Informasi", "sks": 2}
7         ],
8         "Sistem & Teknologi Informasi": [
9             {"matkul": "Dasar-Dasar Sistem Informasi", "sks": 3},
10            {"matkul": "Matriks & Ruang Vektor", "sks": 3},
11            {"matkul": "Manajemen Bisnis Pengantar", "sks": 2}
12        ],
13        "Teknik Arsitektur": [
14            {"matkul": "Pengantar Studio Perancangan", "sks": 4},
15            {"matkul": "Estetika Bentuk", "sks": 2},
16            {"matkul": "Sejarah & Teori Arsitektur I", "sks": 2}
17        ],
18        "Teknik Industri": [
19            {"matkul": "Pengantar Teknik Industri", "sks": 2},
20            {"matkul": "Fisika Dasar I", "sks": 3},
21            {"matkul": "Kalkulus I", "sks": 3}
22        ],
23        "Manajemen": [
24            {"matkul": "Pengantar Manajemen", "sks": 3},
25            {"matkul": "Matematika Ekonomi", "sks": 3},
26            {"matkul": "Pengantar Bisnis", "sks": 3}
27        ],
28        "Akuntansi": [
29            {"matkul": "Pengantar Akuntansi I", "sks": 3},
30            {"matkul": "Matematika Ekonomi", "sks": 3},
31            {"matkul": "Mikroekonomi Pengantar", "sks": 3}
32        ]
33    }
34
35    @classmethod
36    def tampilkan_krs(cls, prodi):
37        print("\n=====")
38        print(f" PAKET KRS SEMESTER 1 - {prodi.upper()} ")
39        print("=====")
40
41        # mengambil daftar matkul berdasarkan prodi
42        matkul_list = cls.JADWAL_PAKET.get(prodi.strip(), [])
43
44        if not matkul_list:
45            print("Belum ada jadwal mata kuliah.")
46            print("=====\n")
47            return
48
49        total_sks = 0
50        for i, item in enumerate(matkul_list, 1):
51            print(f"{i}. {item['matkul']} ({item['sks']} SKS)")
52            total_sks += item['sks']
53
54        print("=====")
55        print(f" TOTAL SKS YANG DIAMBIL: {total_sks} SKS")
56        print("=====\n")
```

Pembahasan Analitis: Fungsi `tampilkan_krs` tidak memerlukan pembuatan objek (*instantiation*) dari kelas `KRS`. Melalui argumen `cls`, metode ini langsung merujuk pada atribut kelas `JADWAL_PAKET`. Loop internal memproses ekstraksi array objek kamus kecil (`item['matkul']` dan `item['sks']`), mengurutkan output menggunakan `enumerate()`, serta menjumlahkan bobot kredit secara akumulatif pada variabel `total_sks` saat runtime.

5.3 Arsitektur Modul Utama (main.py)

Mengatur jalannya loop program serta interaksi pertukaran data antar objek di dalam aplikasi.

```
1  import os
2  from pendaftaran import daftar_maba
3  from penjadwalan import KRS
4
5  # Menyimpan sekumpulan objek Mahasiswa yang mendaftar
6  db_mahasiswa = {}
7
8  def clear_screen():
9      os.system('cls' if os.name == 'nt' else 'clear') # type: ignore
10
11 def login_mhs():
12     clear_screen()
13     print("=== LOGIN PORTAL MAHASISWA ===")
14     username = input("Username (NIM) : ").strip()
15     password = input("Password      : ").strip()
16
17     # Memeriksa password dari atribut objek mahasiswa
18     if username in db_mahasiswa and db_mahasiswa[username].password == password:
19         mhs = db_mahasiswa[username] # Mengambil objek mahasiswa berdasarkan NIM
20
21         clear_screen()
22         print(f"Selamat datang, {mhs.nama}!")
23         print(f"Program Studi: {mhs.prodi}")
24
25         # Memanggil method class KRS
26         KRS.tampilkan_krs(mhs.prodi)
27         input("Tekan Enter untuk keluar dari halaman KRS...")
28     else:
29         print("\n[-] Login Gagal. NIM atau Password salah / belum terdaftar.")
30         input("\nTekan Enter untuk kembali...")
```

Pembahasan Analitis: Baris kode `mhs = db_mahasiswa[username]` membuktikan efisiensi PBO. Ketika login valid, variabel `mhs` memegang kendali penuh atas referensi objek Mahasiswa yang dibuat di modul terpisah. Properti `mhs.nama` dan `mhs.prodi` dapat diakses secara langsung tanpa perlu melakukan proses parsing string manual.

BAB VI

EVALUASI DAN PENGUJIAN SISTEM

6.1 Matriks Skenario Uji (Black Box Testing)

Pengujian sistem dilakukan secara fungsional menggunakan metode *Black Box Testing* untuk menjamin seluruh blok kode percabangan logis tereksekusi dengan benar sesuai masukan pengguna.

ID Uji	Komponen Sistem	Data Masukan (Input)	Respons Sistem (Output Diharapkan)	Hasil Pengujian
TC-01	Menu Pendaftaran Maba	Angka 1 pada Menu Utama	Layar terminal dibersihkan, formulir isian registrasi dibuka secara berurutan.	SESUAI
TC-02	Validasi Fakultas dan Prodi	Pilih Fakultas: 1, Pilih Prodi: 2	Berhasil memetakan pilihan ke Program Studi "Sistem & Teknologi Informasi".	SESUAI
TC-03	Penanganan Kredensial	Tgl Lahir: 30062007	Terbit NIM 10 digit (misal: 2612984531) dan password 30062007.	SESUAI
TC-04	Otentikasi Pengguna	NIM & Password sesuai registrasi	Berhasil melakukan login, menampilkan pesan sambutan personal.	SESUAI
TC-05	Integrasi Penjadwalan	Sukses Login Mahasiswa	Memanggil KRS.tampilkan_krs() dan mencetak rincian total paket SKS.	SESUAI

ID Uji	Komponen Sistem	Data Masukan (Input)	Respons Sistem (Output Diharapkan)	Hasil Pengujian
TC-06	Proteksi Login	Kredensial asal/salah	Menampilkan peringatan kegagalan otentikasi, akses menu KRS ditolak.	SESUAI
TC-07	Otentikasi Admin	User: admin, Pass: admin	Membuka gerbang administrator internal dan menampilkan jumlah agregat.	SESUAI

6.2 Dokumentasi Hasil Eksekusi Terminal Visual Studio Code

6.2.1 Log Tampilan Pendaftaran (Pilihan Menu 1)

```

=====
SISTEM INFORMASI AKADEMIK MAHASISWA BARU
=====
1. Daftar Mahasiswa Baru
2. Login Mahasiswa (Cetak KRS)
3. Menu Admin
4. Keluar Program
=====
Pilih menu (1/2/3/4): █

=====
PENDAFTARAN BERHASIL DISIMPAN
=====
Username (NIM) : 2612787717
Password      : 01012001
=====
Harap catat Username dan Password Anda untuk Login.
Tekan Enter untuk kembali ke Menu Utama...█

=====
=== PENDAFTARAN MAHASISWA BARU ===
Nama Lengkap      : Aditya Edgar Saputra
Jenis Kelamin (L/P) : L
Tempat Lahir      : Sidoarjo
Tanggal Lahir (DDMMYYYY): 01012001
No. Telp          : 089865434356
Alamat Rumah      : Krian
Email             : aditya@gmail.com
Kewarganegaraan   : WNI

--- PILIHAN FAKULTAS ---
1. FTEIC
2. TEKNIK
3. FEB
Pilih Fakultas (1/2/3): 1

--- PILIHAN PRODI FTEIC ---
1. Teknik Informatika
2. Sistem & Teknologi Informasi
Pilih Prodi (1/2): 2

```


6.2.2 Log Tampilan Halaman KRS Mahasiswa (Pilihan Menu 2)

```
=== LOGIN PORTAL MAHASISWA ===
Username (NIM) : 2612787717
Password      : 01012001

Selamat datang, Aditya Edgar Saputra!
Program Studi: Sistem & Teknologi Informasi

=====
PAKET KRS SEMESTER 1 - SISTEM & TEKNOLOGI INFORMASI
=====
1. Dasar-Dasar Sistem Informasi (3 SKS)
2. Matriks & Ruang Vektor (3 SKS)
3. Manajemen Bisnis Pengantar (2 SKS)
=====
TOTAL SKS YANG DIAMBIL: 8 SKS
=====

Tekan Enter untuk keluar dari halaman KRS...
```

6.2.3 Log Tampilan Menu Admin (Pilihan Menu 3)

```
=====
SISTEM INFORMASI AKADEMIK MAHASISWA BARU
=====
1. Daftar Mahasiswa Baru
2. Login Mahasiswa (Cetak KRS)
3. Menu Admin
4. Keluar Program
=====
Pilih menu (1/2/3/4): 3
masukkan username admin:admin
masukkan password admin:admin

=====
MENU ADMINISTRATOR
=====
Total mahasiswa terdaftar: 2 orang

statistik mahasiswa per program studi:
- Sistem & Teknologi Informasi: 2 mahasiswa
=====
Tekan enter untuk kembali ke menu utama...
```

BAB VII PENUTUP

7.1 Kesimpulan

Berdasarkan hasil perancangan, bedah arsitektur kode, serta pengujian fungsional pada Visual Studio Code, penulis menarik beberapa kesimpulan sebagai berikut:

- a. **Implementasi OOP modular** melalui pemisahan file `main.py`, `pendaftaran.py`, dan `penjadwalan.py` terbukti meningkatkan kerapian struktur kode, meminimalkan keterikatan fungsi (*low coupling*), serta mempermudah proses *debugging*.
- b. **Konsep enkapsulasi** melalui *private method* `__generate_nim` pada class Mahasiswa berhasil mengamankan aturan pembentukan NIM dari intervensi luar dan secara andal mencegah terjadinya NIM ganda.
- c. **Penerapan dekorator `@classmethod`** pada class KRS efektif untuk menyajikan data paket kurikulum secara dinamis serta mengotomatisasi kalkulasi beban studi tanpa membebani alokasi memori *runtime*.

7.2 Saran

Meskipun purwarupa aplikasi telah berjalan dengan stabil, pengembangan sistem lebih lanjut dapat dioptimalkan melalui beberapa saran teknis berikut:

- a. **Persistensi Data Permanen:** Mengalihkan penyimpanan data dari *in-memory dictionary* ke RDBMS seperti MySQL atau SQLite agar data tidak terhapus saat aplikasi ditutup.
- b. **Pengembangan GUI:** Mengubah visualisasi antarmuka dari berbasis teks (CLI) menjadi *Graphical User Interface* (GUI) menggunakan pustaka Tkinter atau PyQt5 untuk meningkatkan aspek *usability* dan kenyamanan pengguna.
- c. **Validasi Input Terstruktur:** Menambahkan fungsi *exception handling* menyeluruh untuk memvalidasi tipe data masukan (misalnya membatasi nomor telepon hanya untuk angka) guna mencegah aplikasi berhenti mendadak akibat *human error*.

DAFTAR PUSTAKA

- Connolly, T. M., & Begg, C. E. (2015). *Database systems: A practical approach to design, implementation, and management* (6th ed.). Pearson Education.
- Deitel, P., & Deitel, H. (2019). *Intro to Python for computer science and data science: Learning to program with AI, big data and the cloud*. Pearson.
- Lutz, M. (2013). *Learning Python: Powerful object-oriented programming* (5th ed.). O'Reilly Media.
- Matthes, E. (2023). *Python crash course: A hands-on, project-based introduction to programming* (3rd ed.). No Starch Press.
- Ramalho, L. (2022). *Fluent Python: Clear, concise, and effective programming* (2nd ed.). O'Reilly Media.
- Schildt, H. (2018). *Java: The complete reference* (11th ed.). McGraw-Hill Education.
- Stallings, W. (2018). *Computer organization and architecture: Designing for performance* (11th ed.). Pearson.
- Weisfeld, M. (2019). *The object-oriented thought process* (5th ed.). Addison-Wesley Professional.